

编译原理 2025 风格模拟卷03（题目与考试标准答案）

卷面说明

本卷按 Slides/Slides/handout 与三份 Recap PPT 更新，主线覆盖前端自动机与 LL、中端 IR/SSA、数据流、符号执行、后端目标代码、寄存器分配、指令调度以及过程间/指针分析。总分 100 分。

A 卷题目

一、词法分析（15 分）

正则表达式：

```
(a|b)* a a
```

1. 使用 Thompson 构造法构造等价 NFA。
2. 使用子集构造法构造 DFA，写明 DFA 状态含义。
3. 最小化 DFA。

二、LL 语法分析（15 分）

文法：

```
R -> T U  
T -> T p | id  
U -> l | r
```

1. 消除左递归。
2. 写 FIRST 和 FOLLOW。
3. 写预测分析表。
4. 判断消除左递归后的文法是否为 LL(1)。

三、指令调度（15 分）

给定指令：

```
I1: R1 = R2 + R3  
I2: R4 = R1 + R5  
I3: R2 = R6 + R7  
I4: R1 = R8 + R9  
I5: R10 = R4 + R1
```

1. 写出指令调度的目的。
2. 写出 RAW、WAR、WAW 的定义。
3. 构建依赖图，WAR/WAW 用虚线表示。
4. 在允许寄存器重命名的条件下给出一个合法重排序列。

四、支配问题（15 分）

CFG：双分支汇合后出口

```
Entry->A  
A->B  
A->C  
B->D  
C->D  
D->E  
D->F  
E->Exit  
F->Exit
```

1. 写出 Dom、PostDom、Dom Frontier 的定义。
2. 计算每个节点的 Dom 集和 idom。
3. 写出支配边界。

4. 说明 SSA 中 phi 函数应放在什么位置。

五、符号执行与 SAT (20 分)

公式:

```
F = ((a3 and b3) or c3)
```

1. 写出 Path-sensitive、Flow-sensitive、Context-sensitive 的定义。
2. 写出 CNF 的定义。
3. 使用 Tseitin 算法把 F 转成 CNF。
4. 证明 2025 卷中的两组公式同时可满足。

六、Andersen 指针分析 (20 分)

程序:

```
p = &A
q = &B
r = &C
*p = q
q = r
t = *p
```

1. 对每条语句写 Andersen 约束。
2. 根据约束构建闭包。
3. 写出最终 points-to 集合。

七、PPT 新增重点: SSA 与 phi (10 分)

CFG:

```
B1->B2
B1->B3
B2->B4
B3->B4

B2: x = 1
B3: x = 2
B4: y = x + 1
```

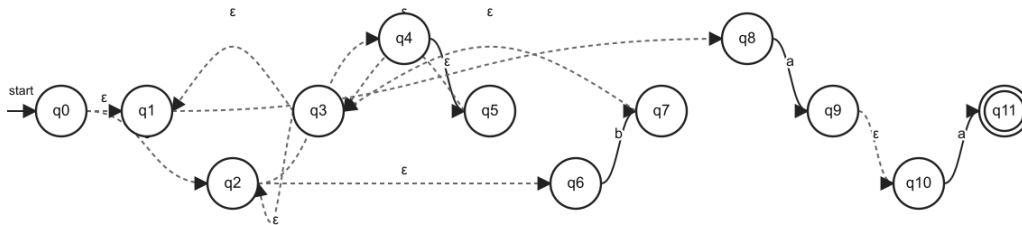
1. 说明 SSA 的基本要求。
2. 判断在哪里需要插入 x 的 phi。
3. 写出 SSA 形式的 B2、B3、B4。

B 按考试标准重写答案

一、词法分析答案

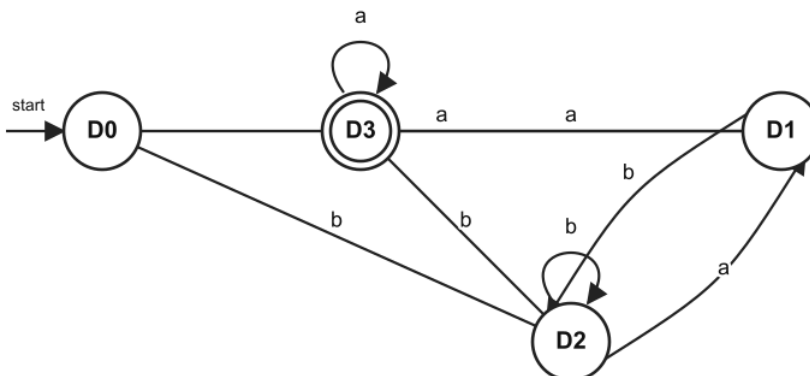
本题图形化答案如下，三张图分别对应 Thompson NFA、子集构造 DFA 和最小 DFA。

Thompson NFA: $(a|b)^* a a$



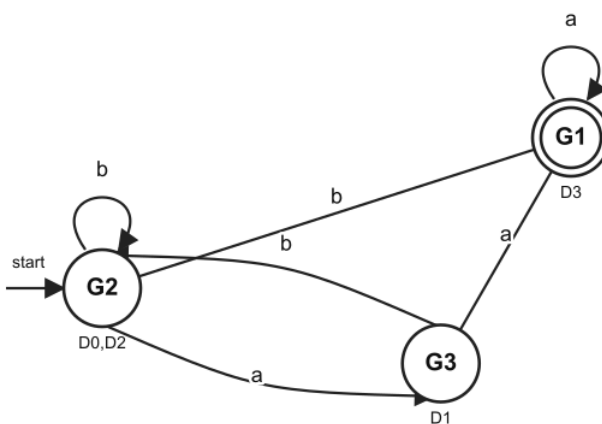
模拟卷03 Thompson NFA

Subset DFA: $(a|b)^* a a$



模拟卷03 子集构造 DFA

Minimal DFA: $(a|b)^* a a$



模拟卷03 最小 DFA

正则表达式: $(a|b)^* a a$

Thompson NFA: 起点 q_0 , 终点 q_{11} 。按下列转移表画图, q_{11} 画双圈。

$q_4 \xrightarrow{a} q_5$
 $q_6 \xrightarrow{b} q_7$
 $q_2 \xrightarrow{\epsilon} q_4$
 $q_2 \xrightarrow{\epsilon} q_6$
 $q_5 \xrightarrow{\epsilon} q_3$
 $q_7 \xrightarrow{\epsilon} q_3$
 $q_0 \xrightarrow{\epsilon} q_2$
 $q_0 \xrightarrow{\epsilon} q_1$
 $q_3 \xrightarrow{\epsilon} q_2$
 $q_3 \xrightarrow{\epsilon} q_1$
 $q_8 \xrightarrow{a} q_9$
 $q_1 \xrightarrow{\epsilon} q_8$
 $q_{10} \xrightarrow{a} q_{11}$
 $q_9 \xrightarrow{\epsilon} q_{10}$

子集构造 DFA:

D0:

NFA集合 = $\{q_0, q_1, q_2, q_4, q_6, q_8\}$

on a $\rightarrow$ D1

on b $\rightarrow$ D2

终态 = 否

D1:

NFA集合 = $\{q_1, q_2, q_3, q_4, q_5, q_6, q_8, q_9, q_{10}\}$

on a $\rightarrow$ D3

on b $\rightarrow$ D2

终态 = 否

D2:

NFA集合 = $\{q_1, q_2, q_3, q_4, q_6, q_7, q_8\}$

on a $\rightarrow$ D1

on b $\rightarrow$ D2

终态 = 否

D3:

NFA集合 = $\{q_1, q_2, q_3, q_4, q_5, q_6, q_8, q_9, q_{10}, q_{11}\}$

on a $\rightarrow$ D3

on b $\rightarrow$ D2

终态 = 是

最小化:

初分组:

终态组 = $\{D3\}$

非终态组 = $\{D0, D1, D2\}$

最终分组:

G1 = $\{D3\}$

G2 = $\{D0, D2\}$

G3 = $\{D1\}$

每个最终分组对应一个最小 DFA 状态。

最小化推理过程:

当前分组	检查状态	输入 a 的目标组	输入 b 的目标组	结论
$\{D3\}$	D3	$\{D3\}$	$\{D0, D1, D2\}$	终态组单独保留
$\{D0, D1, D2\}$	D0	$\{D0, D1, D2\}$	$\{D0, D1, D2\}$	先与 D2 相同
$\{D0, D1, D2\}$	D2	$\{D0, D1, D2\}$	$\{D0, D1, D2\}$	D0、D2 可合并
$\{D0, D1, D2\}$	D1	$\{D3\}$	$\{D0, D1, D2\}$	输入 a 后进入终态组, 与 D0、D2 不同

因此非终态组继续划分为 $\{D0, D2\}$ 和 $\{D1\}$ 。最终最小 DFA 为:

最小 DFA 状态	原 DFA 状态集合	a	b	是否终态
G2	$\{D0, D2\}$	G3	G2	否
G3	$\{D1\}$	G1	G2	否
G1	$\{D3\}$	G1	G2	是

其中 G2 是初态, G1 是终态。该最小 DFA 的含义是: G2 表示当前串末尾不是 a, G3 表示当前串末尾有一个 a, G1 表示当前串末尾是 aa。

二、LL 语法分析答案

原文法：

$R \rightarrow T U$
 $T \rightarrow T p \mid id$
 $U \rightarrow l \mid r$

消除左递归：

$R \rightarrow T U$
 $T \rightarrow id T'$
 $T' \rightarrow p T' \mid \epsilon$
 $U \rightarrow l \mid r$

FIRST:

$FIRST(R) = \{ id \}$
 $FIRST(T) = \{ id \}$
 $FIRST(T') = \{ p, \epsilon \}$
 $FIRST(U) = \{ l, r \}$

FOLLOW:

$FOLLOW(R) = \{ \$ \}$
 $FOLLOW(T) = \{ l, r \}$
 $FOLLOW(T') = \{ l, r \}$
 $FOLLOW(U) = \{ \$ \}$

预测表：

$M[R, id] = R \rightarrow T U$
 $M[T, id] = T \rightarrow id T'$
 $M[T', p] = T' \rightarrow p T'$
 $M[T', l] = T' \rightarrow \epsilon$
 $M[T', r] = T' \rightarrow \epsilon$
 $M[U, l] = U \rightarrow l$
 $M[U, r] = U \rightarrow r$

结论：表中无多重入口，消除左递归后的文法是 LL(1)。

左递归消除推导：

$T \rightarrow T p \mid id$

套用直接左递归消除规则：

$A \rightarrow A \alpha \mid \beta$

改写为：

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \epsilon$

所以：

$\alpha = p$

$\beta = id$

$T \rightarrow id T'$

$T' \rightarrow p T' \mid \epsilon$

FIRST 推导：

符号	推导依据	FIRST	
T	$T \rightarrow id$ T' ，右部第一个符号是终结符 id	$\{ id \}$	
T'	$T' \rightarrow p T'$	ϵ	$\{ p, \epsilon \}$
U	$U \rightarrow l$	r	$\{ l, r \}$
R	$R \rightarrow T U$ ，且 T 不能推出 ϵ	$\{ id \}$	

FOLLOW 推导：

FOLLOW 集	推导依据	结果
$FOLLOW(R)$	R 是开始符号	$\{ \$ \}$
$FOLLOW(T)$	$R \rightarrow T U$ ，T 后面是 U，所以加入 $FIRST(U)$	$\{ l, r \}$

FOLLOW 集	推导依据	结果
FOLLOW(T')	$T \rightarrow id\ T'$, T' 在产生式末尾, 所以加入 FOLLOW(T)	{l, r}
FOLLOW(U)	$R \rightarrow T\ U$, U 在产生式末尾, 所以加入 FOLLOW(R)	{ \$ }

预测分析表填表推导:

产生式	FIRST 或 FOLLOW 依据	填入表项
$R \rightarrow T\ U$	$FIRST(T\ U) = FIRST(T) = \{id\}$	$M[R, id] = R \rightarrow T\ U$
$T \rightarrow id\ T'$	$FIRST(id\ T') = \{id\}$	$M[T, id] = T \rightarrow id\ T'$
$T' \rightarrow p\ T'$	$FIRST(p\ T') = \{p\}$	$M[T', p] = T' \rightarrow p\ T'$
$T' \rightarrow \epsilon$	$FOLLOW(T') = \{l, r\}$	$M[T', l] = T' \rightarrow \epsilon$, $M[T', r] = T' \rightarrow \epsilon$
$U \rightarrow l$	$FIRST(l) = \{l\}$	$M[U, l] = U \rightarrow l$
$U \rightarrow r$	$FIRST(r) = \{r\}$	$M[U, r] = U \rightarrow r$

预测分析表 (表格形式):

非终结符	输入符号	产生式
R	id	$R \rightarrow T\ U$
T	id	$T \rightarrow id\ T'$
T'	p	$T' \rightarrow p\ T'$
T'	l	$T' \rightarrow \epsilon$
T'	r	$T' \rightarrow \epsilon$
U	l	$U \rightarrow l$
U	r	$U \rightarrow r$

完整预测分析表, 空白项为 error:

非终结符	id	p	l	r	\$
R	$R \rightarrow T\ U$	error	error	error	error
T	$T \rightarrow id\ T'$	error	error	error	error
T'	error	$T' \rightarrow p\ T'$	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$	error
U	error	error	$U \rightarrow l$	$U \rightarrow r$	error

每个表项最多只有一条产生式, 因此消除左递归后的文法是 LL(1)。

三、指令调度答案

定义:

指令调度在不改变程序语义的前提下重排指令, 减少流水线停顿, 提高并行执行效率。
 RAW: 先写后读, 必须保持顺序。
 WAR: 先读后写, 写者不能提前。
 WAW: 先写后写, 顺序影响最终值。

指令:

I1: $R1 = R2 + R3$
 I2: $R4 = R1 + R5$
 I3: $R2 = R6 + R7$
 I4: $R1 = R8 + R9$
 I5: $R10 = R4 + R1$

依赖:

I1 $\rightarrow$ I2 RAW on R1
 I1 $\rightarrow$ I3 WAR on R2
 I1 $\rightarrow$ I4 WAW on R1

I2 -> I4 WAR on R1
 I2 -> I5 RAW on R4
 I4 -> I5 RAW on R1

寄存器重命名：
 I3: R2_new = R6 + R7
 I4: R1_new = R8 + R9
 I5 使用 R1_new

重命名后保留 RAW，WAR/WAW 被消除。一个合法调度序列：
 I1, I3, I4, I2, I5

先列每条指令的读写集合：

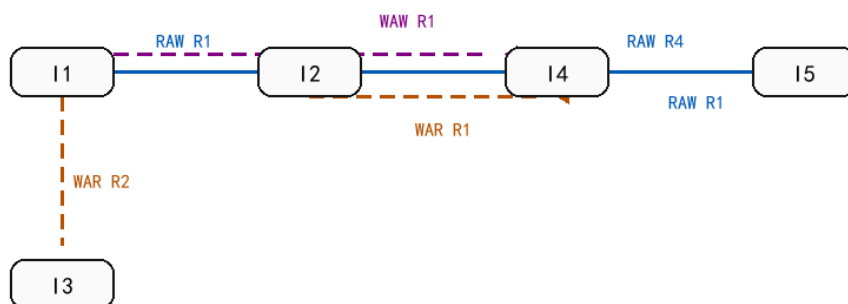
指令	读寄存器	写寄存器
I1	R2, R3	R1
I2	R1, R5	R4
I3	R6, R7	R2
I4	R8, R9	R1
I5	R4, R1	R10

逐对检查依赖：

依赖边	类型	寄存器	判断逻辑
I1 -> I2	RAW	R1	I1 写 R1, I2 读 R1, I2 必须读到 I1 的结果
I1 -> I3	WAR	R2	I1 读 R2, I3 后写 R2, I3 不能提前破坏 I1 的读值
I1 -> I4	WAW	R1	I1 写 R1, I4 后写 R1, 两个写入顺序影响最终版本
I2 -> I4	WAR	R1	I2 读 R1, I4 后写 R1, I4 不能提前破坏 I2 的读值
I2 -> I5	RAW	R4	I2 写 R4, I5 读 R4
I4 -> I5	RAW	R1	I4 写 R1, I5 读 R1, I5 需要 I4 的结果

依赖图如下。实线是 RAW，虚线是 WAR/WAW：

模拟卷03 指令依赖图：实线 RAW，虚线 WAR/WAW



寄存器重命名后，WAR/WAW 虚线约束可消除；RAW 实线约束仍必须保留。

模拟卷03 指令依赖图

寄存器重命名后：

原指令	重命名后
I3: R2 = R6 + R7	I3: R2_new = R6 + R7
I4: R1 = R8 + R9	I4: R1_new = R8 + R9
I5: R10 = R4 + R1	I5: R10 = R4 + R1_new

重命名消除了 WAR/WAW，仍必须满足：

```
I1 -> I2
I2 -> I5
I4 -> I5
```

序列 I1, I3, I4, I2, I5 的合法性检查：

约束	在该序列中的相对顺序	是否满足
I1 -> I2	I1 在 I2 前	是
I2 -> I5	I2 在 I5 前	是
I4 -> I5	I4 在 I5 前	是

所以 I1, I3, I4, I2, I5 是寄存器重命名后的一个合法调度序列。

四、支配问题答案

定义：

Dom(n)：入口到 n 的每条路径都经过的节点集合。
 PostDom(n)：n 到出口的每条路径都经过的节点集合。
 Dom Frontier(n)：n 支配某个前驱，但不严格支配该节点的位置。

CFG 边：
 Entry->A
 A->B
 A->C
 B->D
 C->D
 D->E
 D->F
 E->Exit
 F->Exit

Dom 集：
 Dom(Entry)={Entry}
 Dom(A)={Entry,A}
 Dom(B)={Entry,A,B}
 Dom(C)={Entry,A,C}
 Dom(D)={Entry,A,D}
 Dom(E)={Entry,A,D,E}
 Dom(F)={Entry,A,D,F}
 Dom(Exit)={Entry,A,D,Exit}

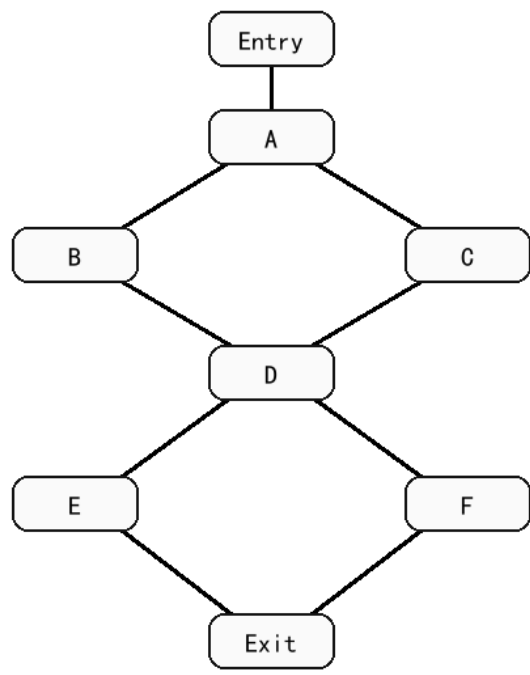
直接支配者：
 idom(A)=Entry
 idom(B)=A
 idom(C)=A
 idom(D)=A
 idom(E)=D
 idom(F)=D
 idom(Exit)=D

支配边界：
 DF(B)={D}
 DF(C)={D}
 DF(E)={Exit}
 DF(F)={Exit}
 其余节点 DF 为空集

SSA phi 放置：若变量在多个基本块中被定义，则在这些定义块的迭代支配边界处插入该变量的 phi。直观上，phi 放在多条控制流汇合且不同定义可能到达的位置。

CFG 图如下：

模拟卷03 双分支汇合 CFG



模拟卷03 双分支汇合 CFG

Dom 集按方程计算：

```
Dom(Entry) = {Entry}
Dom(n) = {n} union intersection(Dom(p)), p 是 n 的所有前驱
```

逐点代入：

节点	前驱	计算过程	Dom 集
Entry	无	入口节点固定	{Entry}
A	Entry	{A} union Dom(Entry)	{Entry,A}
B	A	{B} union Dom(A)	{Entry,A,B}
C	A	{C} union Dom(A)	{Entry,A,C}
D	B, C	{D} union (Dom(B) intersection Dom(C))	{Entry,A,D}
E	D	{E} union Dom(D)	{Entry,A,D,E}
F	D	{F} union Dom(D)	{Entry,A,D,F}
Exit	E, F	{Exit} union (Dom(E) intersection Dom(F))	{Entry,A,D,Exit}

直接支配者 idom：

节点	严格支配者	idom
A	{Entry}	Entry
B	{Entry,A}	A
C	{Entry,A}	A
D	{Entry,A}	A
E	{Entry,A,D}	D
F	{Entry,A,D}	D
Exit	{Entry,A,D}	D

支配边界按定义判断：若节点 X 支配 Y 的某个前驱，但 X 不严格支配 Y，则 Y 属于 DF(X)。

X	检查汇合点	判断	DF(X)
B	D	B 支配 D 的前驱 B，但 B 不严格支配 D	{D}
C	D	C 支配 D 的前驱 C，但 C 不严格支配 D	{D}
E	Exit	E 支配 Exit 的前驱 E，但 E 不严格支配 Exit	{Exit}
F	Exit	F 支配 Exit 的前驱 F，但 F 不严格支配 Exit	{Exit}
Entry, A, D	D 或 Exit	它们严格支配对应汇合点	{}

因此若变量在 B、C 两个分支中分别定义，phi 放在 D；若变量在 E、F 两个分支中分别定义，phi 放在 Exit。

五、符号执行与 SAT 答案

公式：F = ((a3 and b3) or c3) 引入：x31 <-> (a3 and b3), x32 <-> (x31 or c3)，根变量 x32 为真。 CNF: (not x31 or a3) (not x31 or b3) (x31 or not a3 or not b3) (x32 or not x31) (x32 or not c3) (not x32 or x31 or c3) (x32) 敏感性定义： Path-sensitive 区分不同路径。 Flow-sensitive 区分程序点和语句顺序。 Context-sensitive 区分函数调用上下文。 同时可满足证明套用： F1=(a0 or ... or an or c) and (b0 or ... or bm or not c), F2=(a0 or ... or an or b0 or ... or bm)。 F1 为真时，c=false 推出某个 ai=true, c=true 推出某个 bj=true, 所以 F2 为真。 F2 为真时，若某个 ai=true 令 c=false; 若某个 bj=true 令 c=true, 所以 F1 为真。

Tseitin 转换的推导表：

子公式	新变量	等价关系	CNF 子句
a3 and b3	x31	x31 <-> (a3 and b3)	(not x31 or a3); (not x31 or b3); (x31 or not a3 or not b3)
x31 or c3	x32	x32 <-> (x31 or c3)	(x32 or not x31); (x32 or not c3); (not x32 or x31 or c3)
整个公式为真	x32	根变量取真	(x32)

所以最终 CNF 是：

(not x31 or a3) (not x31 or b3) (x31 or not a3 or not b3) (x32 or not x31) (x32 or not c3) (not x32 or x31 or c3) (x32)
--

两组公式同时可满足的证明按两个方向写：

已知	分情况	构造或推出	结论
F1=(a0 or ... or an or c) and (b0 or ... or bm or not c) 为真	c=false	第一项要为真，必须有某个 ai=true	F2 为真

已知	分情况	构造或推出	结论
F1 为真	$c = \text{true}$	第二项要为真，必须有某个 $b_j = \text{true}$	F2 为真
$F2 = (a_0 \text{ or } \dots \text{ or } a_n \text{ or } b_0 \text{ or } \dots \text{ or } b_m)$ 为真	存在某个 $a_i = \text{true}$	取 $c = \text{false}$	F1 为真
F2 为真	存在某个 $b_j = \text{true}$	取 $c = \text{true}$	F1 为真

因此两组公式存在满足赋值的条件一致，可同时满足。

六、Andersen 指针分析答案

程序:

```
p = &A
q = &B
r = &C
*p = q
q = r
t = *p
```

约束:

```
A in pts(p)
B in pts(q)
C in pts(r)
for v in pts(p), pts(q) subset pts(v)
pts(r) subset pts(q)
for v in pts(p), pts(v) subset pts(t)
```

闭包:

```
pts(p)={A}
pts(q)={B,C}
pts(r)={C}
由 *p=q 得 pts(A)={B,C}
由 t=*p 得 pts(t)={B,C}
```

逐条语句生成约束:

语句	Andersen 约束	含义
$p = \&A$	$A \text{ in } \text{pts}(p)$	p 指向 A
$q = \&B$	$B \text{ in } \text{pts}(q)$	q 指向 B
$r = \&C$	$C \text{ in } \text{pts}(r)$	r 指向 C
$*p = q$	对每个 $v \text{ in } \text{pts}(p)$, $\text{pts}(q) \text{ subset } \text{pts}(v)$	q 指向的对象流入 p 所指对象
$q = r$	$\text{pts}(r) \text{ subset } \text{pts}(q)$	r 的指向集合流入 q
$t = *p$	对每个 $v \text{ in } \text{pts}(p)$, $\text{pts}(v) \text{ subset } \text{pts}(t)$	p 所指对象的内容流入 t

闭包计算过程:

轮次	新增事实	原因
0	$\text{pts}(p) = \{A\}$, $\text{pts}(q) = \{B\}$, $\text{pts}(r) = \{C\}$	三条取地址语句
1	$\text{pts}(q)$ 加入 C	$q = r$, 所以 $\text{pts}(r) \text{ subset } \text{pts}(q)$
2	$\text{pts}(A)$ 加入 B,C	$*p = q$, 且 $\text{pts}(p) = \{A\}$ 、 $\text{pts}(q) = \{B,C\}$
3	$\text{pts}(t)$ 加入 B,C	$t = *p$, 且 $\text{pts}(p) = \{A\}$ 、 $\text{pts}(A) = \{B,C\}$

最终 points-to 集合:

符号	points-to 集合
p	{A}
q	{B,C}
r	{C}
t	{B,C}

符号	points-to 集合
A	{B,C}
B	{}
C	{}

七、PPT 新增重点答案

SSA: Static Single Assignment, 每个变量只有一个定义。
 在控制流汇合处, 如果同一个变量可能来自多个定义, 需要插入 phi。

x 在 B2 和 B3 中分别定义, 二者在 B4 汇合。
 因此在 B4 插入 x 的 phi。

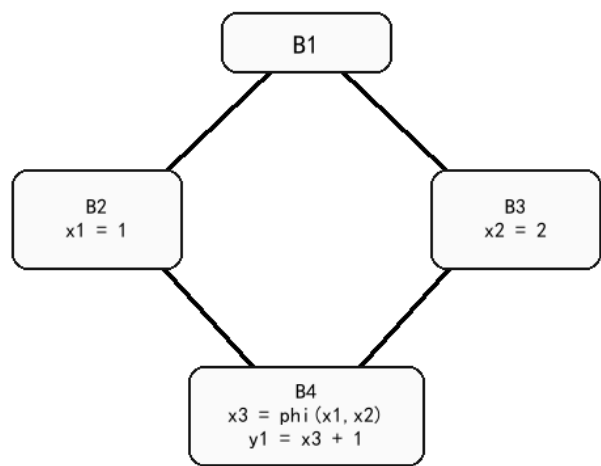
B2:
 $x1 = 1$

B3:
 $x2 = 2$

B4:
 $x3 = \text{phi}(x1 \text{ from B2}, x2 \text{ from B3})$
 $y1 = x3 + 1$

SSA 与 phi 插入图如下:

模拟卷03 SSA phi 插入位置



模拟卷03 SSA phi 插入位置

推理过程:

基本块	原语句	SSA 改写	原因
B2	$x = 1$	$x1 = 1$	x 的第一个定义改名为 x1
B3	$x = 2$	$x2 = 2$	x 的另一个定义改名为 x2
B4	$y = x + 1$	先插入 $x3 = \text{phi}(x1 \text{ from B2}, x2 \text{ from B3})$, 再写 $y1 = x3 + 1$	B4 有来自 B2、B3 的两条路径, x 的到达定义不同

插入 phi 的依据:

B2 和 B3 都定义 x。
 B4 是 B2、B3 的控制流汇合点。
 在 B4 使用 x 时, 运行时可能来自 B2 的 x1, 也可能来自 B3 的 x2。

因此 B4 必须先用 phi 合并两个定义。